TP1 : Cryptographie, certificats et confiance

1. Contexte et mission

J'ai démontré expérimentalement qu'un hash garantit l'intégrité mais pas l'identité, qu'un certificat signé n'est pas forcément approuvé, et proposé une PKI à deux niveaux pour ALPINA SERVICES.

ALPINA SERVICES (450 postes, 35 serveurs) a subi un incident : un fichier de configuration modifié sans détection et des alertes de certificat ignorées. Le projet « Confiance numérique 2026 » me demande, avant de déployer AD CS, de distinguer intégrité, chiffrement, signature et confiance.

Critère de réussite	Partie	Statut
Prouver qu'une modification change l'empreinte SHA-256	A	Atteint
Générer une clé et une CSR sans divulguer la clé privée	B	Atteint
Créer et inspecter un certificat autosigné avec SAN DNS	C	Atteint
Expliquer pourquoi un certificat valide n'est pas forcément approuvé	C, D	Atteint
Proposer une architecture Root CA / Issuing CA	E	Atteint

Environnement : Windows 11, PowerShell 5.1, OpenSSL 3.5.5 (fourni par Git for Windows, ajouté au PATH de la session), répertoire C:\TP-PKI\TP1, nom fictif intranet.alpina.test.

Règles respectées : lab.key jamais affichée ni transmise, aucune racine importée dans le magasin de confiance, aucune validation TLS désactivée, aucune modification de l'horloge système.

Capture : Captures\0_environnement.png

2. Partie A : Qualification d'un incident d'intégrité (INC-2048)

Un seul octet modifié sur 27 change 122 des 256 bits de l'empreinte SHA-256 : l'exemplaire initial est celui dont le hash vaut b9568c60…f144f8ff.

Q1-Q2 : Empreinte de preuve.txt avec trois outils

Le fichier preuve.txt contient « ALPINA - configuration v1 » et pèse 27 octets : 25 caractères plus le retour à la ligne Windows (CRLF, 0D 0A).

Outil	Empreinte SHA-256
Get-FileHash (PowerShell)	B9568C6046A39C5D0AA41148CB646A813EA81C0CD5027E9431051367F144F8FF
certutil -hashfile (Windows)	b9568c6046a39c5d0aa41148cb646a813ea81c0cd5027e9431051367f144f8ff
openssl dgst -sha256	b9568c6046a39c5d0aa41148cb646a813ea81c0cd5027e9431051367f144f8ff

Les trois valeurs sont identiques (seule la casse diffère). SHA-256 est normalisé et déterministe : le résultat ne dépend ni de l'outil ni de la machine. Le même contenu recréé sur une autre machine a donné exactement la même empreinte.

Captures : Captures\A1_creation_preuve.png ; Captures\A2_hash_v1_trois_outils.png

Q3 : Modification d'un seul caractère

Une copie de l'original a été conservée (preuve_original.txt), puis « v1 » a été remplacé par « v2 » dans preuve.txt.

Fichier	Taille	Empreinte SHA-256
preuve_original.txt (v1)	27 octets	B9568C6046A39C5D0AA41148CB646A813EA81C0CD5027E9431051367F144F8FF
preuve.txt (v2)	27 octets	2CC4124C9CE40A916A3173088D3A46149C520EC83C569273C5ABD8DAD249EAB9

Format-Hex montre qu'un seul octet diffère, à l'offset 0x18 : 0x31 (« 1 ») devient 0x32 (« 2 »), soit 2 bits modifiés sur 216. Pourtant 122 bits sur 256 de l'empreinte changent (environ 48 %) : c'est l'**effet avalanche**.

Réponse au ticket : l'exemplaire dont l'empreinte correspond à la référence b9568c60… est la version initiale. Les dates de modification (15:04 et 15:11) donnent un indice, mais elles se falsifient facilement et n'entrent pas dans le calcul du hash : seule la comparaison avec une empreinte de référence protégée fait foi.

Captures : Captures\A3_hash_v1_vs_v2.png ; Captures\A3_format_hex.png

Q4 : Pourquoi un hash ne prouve pas l'auteur

Un hash ne fait intervenir aucun secret : n'importe qui peut le recalculer. Je l'ai démontré en jouant l'attaquant : fichier modifié (« configuration PIRATEE »), empreinte officielle recalculée et remplacée, puis contrôle de la victime qui répond True. La falsification passe inaperçue.

Mécanisme complémentaire : la signature numérique. L'auteur signe le hash du fichier avec sa clé privée ; le destinataire vérifie avec la clé publique contenue dans un certificat émis par une AC de confiance. Sans la clé privée, l'attaquant ne peut pas produire de signature valide. On obtient intégrité, authenticité et non-répudiation (par exemple Set-AuthenticodeSignature pour les scripts PowerShell). Le HMAC est une alternative, mais sa clé partagée ne permet pas la non-répudiation.

Capture : Captures\A4_attaque_hash_sans_secret.png

3. Partie B : Identité cryptographique et demande de certificat

Une clé RSA 3072 bits a été générée et protégée, puis une CSR signée pour intranet.alpina.test a été produite et vérifiée, sans jamais exposer la clé privée.

Q5 : Clé privée RSA 3072 bits

La clé a été générée avec openssl genpkey dans lab.key, puis inspectée avec openssl pkey -noout -text_pub. Cette option n'affiche que la partie publique : Public-Key: (3072 bit), un modulus commençant par 00:b4:88:ca:77… et l'exposant 65537 (0x10001). Le contenu de la clé privée n'a jamais été affiché ni capturé.

RSA 3072 bits offre un niveau de sécurité d'environ 128 bits, recommandé pour une utilisation au-delà de 2030.

Protection de la clé : icacls a supprimé l'héritage des droits NTFS sur lab.key et n'accorde plus la lecture/écriture qu'au seul compte de l'administrateur. En production, on ajouterait une passphrase (-aes256), ou on générerait la clé non exportable dans le magasin Windows ou un HSM.

Captures : Captures\B5_cle_publique.png ; Captures\B5bis_protection_cle.png

Q6-Q7 : Génération et inspection de la CSR

Élément	Valeur relevée
Subject	CN=intranet.alpina.test, O=ALPINA SERVICES
SAN (extension demandée)	DNS:intranet.alpina.test
Algorithme de clé	RSA (rsaEncryption), 3072 bits, exposant 65537
Signature de la requête	sha256WithRSAEncryption
Vérification	Certificate request self-signature verify OK

Le modulus de la CSR est identique à celui de lab.key, et la signature fait 384 octets (3072 bits) : la requête contient bien ma clé publique et a été signée par ma clé privée. La vérification réussie prouve que la CSR n'a pas été altérée et que le demandeur possède la clé privée (**preuve de possession**). Elle ne prouve pas que le demandeur a le droit d'utiliser ce nom : ce contrôle revient à l'AC.

Captures : Captures\B6_generation_csr.png ; Captures\B7_csr_inspection.png ; Captures\B7_csr_verify.png

Q8 : Fichiers transmissibles et fichiers protégés

Fichier	Contenu	Transmissible ?
intranet.csr	Clé publique + identité demandée, signées	Oui : seul fichier envoyé à l'AC
intranet.crt	Certificat signé par l'émetteur	Oui : public par nature
lab.key	Clé privée	**Jamais** : elle reste sur le serveur qui l'a générée

L'AC n'a pas besoin de la clé privée : elle signe la clé publique de la CSR. Quiconque obtiendrait lab.key pourrait se faire passer pour intranet.alpina.test. Elle est donc protégée par des droits NTFS restrictifs et exclue de l'archive de preuves.

4. Partie C : Certificat X.509 : création, lecture et limites

Le certificat autosigné est cryptographiquement correct mais rejeté dès qu'on ne le désigne pas soi-même comme racine : OpenSSL renvoie l'erreur 18 et Windows CERT_E_UNTRUSTEDROOT.

Q9 : Création

openssl x509 -req -signkey lab.key -days 30 -sha256 -copy_extensions copy a transformé la CSR en certificat signé par ma propre clé (autosigné). OpenSSL a d'abord revérifié la signature de la CSR (self-signature ok). L'option -copy_extensions copy a conservé le SAN.

Capture : Captures\C9_creation_certificat.png

Q10 : Champs du certificat

Champ	Valeur relevée	Rôle
Subject	CN=intranet.alpina.test, O=ALPINA SERVICES	Identité du titulaire
Issuer	CN=intranet.alpina.test, O=ALPINA SERVICES	Émetteur ; identique au Subject, donc autosigné
SAN	DNS:intranet.alpina.test	Nom réellement contrôlé par les navigateurs (le CN est ignoré)
Not Before / Not After	du 19/10/2026 13:29:00 GMT au 18/11/2026 13:29:00 GMT	Période de validité de 30 jours
Signature Algorithm	sha256WithRSAEncryption	Algorithme de signature de l'émetteur
Public Key Algorithm	rsaEncryption, 3072 bits, exposant 65537	Clé publique du titulaire
Basic Constraints / EKU	**Absents** (ainsi que Key Usage, CDP, AIA)	Voir conséquences ci-dessous

Autres valeurs : version 3, numéro de série 50:4E:FD:FC:E2:99:DA:B7:01:14:35:16:E1:07:98:C4:60:B8:90:53, empreinte SHA-256 52:AD:EE:FD:…:42:FC:44:23. Seules deux extensions sont présentes : le SAN et le Subject Key Identifier, ajouté automatiquement par OpenSSL 3.5. Les heures sont en GMT : 13:29 GMT correspond à 15:29 à Paris.

Conséquences des extensions absentes :

- **Basic Constraints** : rien n'indique si le certificat est une AC (CA:TRUE) ou une entité finale (CA:FALSE) ; les logiciels l'interprètent de façon variable. Un certificat serveur doit porter CA:FALSE.
- **Key Usage / EKU** : aucune restriction d'usage ; le certificat n'est pas limité à l'authentification serveur (serverAuth). Le moindre privilège n'est pas respecté.
- **CDP / AIA** : aucune vérification de révocation possible. Si lab.key était compromise, ce certificat ne pourrait pas être révoqué.

Un certificat émis par AD CS avec le modèle « Serveur Web » contiendrait ces extensions.

Captures : Captures\C10_resume_certificat.png ; Captures\C10_certificat_detail.png

Q11 : Vérification et limites de la confiance

Commande	Résultat
openssl verify -CAfile intranet.crt intranet.crt	intranet.crt: OK
openssl verify intranet.crt	error 18 at 0 depth lookup: self-signed certificate
certutil -verify intranet.crt	CERT_TRUST_IS_SELF_SIGNED, CERT_TRUST_IS_UNTRUSTED_ROOT, 0x800b0109 CERT_E_UNTRUSTEDROOT, révocation impossible à vérifier

Le premier OK est **circulaire** : j'ai moi-même désigné le certificat comme point de confiance, il se porte garant de lui-même. N'importe qui peut fabriquer un certificat autosigné au même nom, avec une signature tout aussi valide. Sans cette désignation, OpenSSL et le moteur de validation de Windows le rejettent.

La confiance ne vient donc pas de la signature mais de la **présence de la racine dans le magasin de confiance** du client. Importer ce certificat sur chaque poste est interdit ici ; c'est le rôle de la PKI AD CS de fournir une racine unique distribuée par GPO.

À noter : la dernière ligne de certutil (« La commande s'est terminée correctement ») signifie que l'outil a fonctionné, pas que le certificat est valide. L'empreinte 0840b4bf… affichée par Windows est l'empreinte SHA-1 du même certificat.

Captures : Captures\C11_verify_confiance.png ; Captures\C11bis_certutil_verify.png

5. Partie D : Diagnostic de trois incidents

Les trois incidents ont été reproduits en laboratoire avec OpenSSL, sans rien importer ni modifier dans Windows : chacun produit un code d'erreur distinct (18, 62 et 10).

	SEC-01 : Racine inconnue	**SEC-02 : SAN non concordant**	**SEC-03 : Certificat expiré**
Symptôme	« Connexion non privée » (NET::ERR_CERT_AUTHORITY_INVALID)	Alerte de nom non concordant (ERR_CERT_COMMON_NAME_INVALID) malgré une chaîne approuvée	Alerte de certificat expiré (ERR_CERT_DATE_INVALID)
Cause probable	Racine absente du magasin « Autorités racines de confiance », ou certificat intermédiaire non envoyé par le serveur	Certificat émis pour serveur.alpina.test et utilisé sur intranet.alpina.test	Date Not After dépassée : renouvellement oublié, pas de supervision (plus rarement, horloge du client erronée)
Contrôle technique	openssl verify : **error 18** ; certutil -verify : **0x800b0109**	openssl x509 -ext subjectAltName puis openssl verify -verify_hostname : **error 62 hostname mismatch**	openssl x509 -enddate ; -checkend : « Certificate will expire » ; openssl verify -attime (01/12/2026) : **error 10 certificate has expired**
Correction attendue	Distribuer la racine d'entreprise par GPO, configurer la chaîne complète sur IIS, publier l'AIA ; jamais « continuer quand même »	Réémettre un certificat dont le SAN contient intranet.alpina.test (plusieurs noms possibles), ou corriger l'URL / le DNS	Renouveler et relier le certificat dans IIS ; auto-enrollment AD CS, supervision des expirations, synchronisation NTP

Dans SEC-02, la vérification avec le nom serveur.alpina.test répond OK : le certificat est approuvé et correctement signé, mais ne prouve pas l'identité du serveur demandé. Dans SEC-03, c'est le même intranet.crt, valide aujourd'hui, qui est rejeté au 1er décembre. La date a été simulée avec -attime pour ne pas modifier l'horloge du poste.

Captures : Captures\C11_verify_confiance.png (SEC-01) ; Captures\C11bis_certutil_verify.png (SEC-01) ; Captures\D_SEC02_san_mismatch.png ; Captures\D_SEC03_expiration.png

Question transversale : signé, valide, identité confirmée

Ces trois notions sont trois niveaux de vérification successifs ; un seul maillon manquant fait tout échouer.

1. **Certificat signé** : une clé privée a signé le contenu. C'est une simple cohérence mathématique ; un certificat autosigné est signé (Q11 : OK), et n'importe qui peut en fabriquer un.
2. **Certificat valide** : signature correcte **et** chaîne jusqu'à une racine approuvée **et** date dans la période de validité **et** non révoqué **et** usages (EKU) adaptés. SEC-01 échoue sur la confiance, SEC-03 sur la date.
3. **Identité du serveur confirmée** : certificat valide **et** nom demandé présent dans le SAN **et** serveur prouvant, lors du handshake TLS, qu'il détient la clé privée. SEC-02 montre qu'un certificat valide peut ne pas confirmer l'identité.

C'est pourquoi une alerte de certificat ne doit jamais être ignorée, contrairement à ce qui s'est produit chez ALPINA SERVICES.

6. Partie E : Architecture cible et restitution au DSI

Je recommande une PKI à deux niveaux : une Root CA hors ligne qui ne signe que l'AC émettrice, et une Issuing CA jointe au domaine qui émet les certificats au quotidien.

Q12-Q13 : Schéma d'architecture et flux

Capture : Captures\E12_architecture_PKI.png (schéma draw.io)

Flux exigé	**N°**	**Description**
Chaîne de certification	1	La Root CA signe le certificat de l'Issuing CA (transfert hors ligne)
Émission	2	L'Issuing CA signe le certificat du serveur IIS (CSR, modèle « Serveur Web »)

Flux exigé	N°	Description
Publication CRL	3	CRL et certificats d'AC publiés sur http://pki.alpina.test (copie manuelle pour la racine)
Demande HTTPS	4	Le client se connecte ; IIS présente son certificat et la chaîne
Contrôle de révocation	5	Le client télécharge la CRL et, si besoin, les certificats d'AC (AIA)
Distribution de la confiance	6	La racine est déployée sur tous les postes par GPO

Q14 : Rôle des autorités et isolement de la racine

La **Root CA** est l'ancre de confiance de toute la PKI. Elle ne signe que le certificat de l'Issuing CA et ses propres CRL. Sa compromission invaliderait tous les certificats de l'entreprise ; elle est donc **isolée** : hors ligne, hors domaine, éteinte, rallumée seulement pour signer une sous-AC ou republier sa CRL (validité longue, par exemple 6 mois).

L'**Issuing CA**, jointe au domaine (Enterprise CA), émet au quotidien les certificats IIS, postes et, demain, Wi-Fi, grâce aux modèles et à l'auto-enrollment par GPO. Elle publie ses CRL et son certificat (AIA) en HTTP. Si elle était compromise, on la révoquerait depuis la racine et on la reconstruirait sans redéployer de racine sur les 450 postes.

Limites d'une seule AC racine d'entreprise (laboratoire simplifié) : racine en ligne et membre du domaine, donc exposée au réseau et à la compromission d'un compte administrateur ; aucune séparation des rôles ; impossible de révoquer l'AC émettrice puisqu'elle est la racine ; une compromission impose de reconstruire toute la PKI. Acceptable en laboratoire, à proscrire en production.

Q15 : Trame de la restitution orale (3 minutes)

Temps	Contenu
0:00 – 0:30	**Contexte** : fichier modifié sans détection, alertes ignorées ; objectif : distinguer intégrité, signature et confiance
0:30 – 1:15	**Erreur détectée** : le certificat autosigné est OK uniquement parce qu'on l'a désigné comme racine ; sinon error 18 et 0x800b0109 ; Basic Constraints, EKU et CDP absents. Signé ne veut pas dire approuvé
1:15 – 2:00	**Bonne pratique** : la clé privée ne quitte jamais le serveur, seule la CSR part vers l'AC ; lab.key protégée par NTFS et exclue des preuves
2:00 – 2:45	**Décision d'architecture** : racine hors ligne + AC émettrice jointe au domaine, racine diffusée par GPO, CRL publiées en HTTP (montrer le schéma)
2:45 – 3:00	**Conclusion** : les trois risques réduits par la PKI

7. Conclusion métier : trois risques que la PKI doit réduire

1. **Altération non détectée de fichiers et de scripts** (cause de l'incident initial) : la signature numérique avec des certificats d'entreprise garantit intégrité et origine, là où un hash seul est contournable (partie A).
2. **Usurpation de serveurs et interception des échanges** (intranet RH, application d'interventions) : des certificats TLS émis par l'AC interne, avec un SAN correct et une révocation possible, permettent aux postes de vérifier réellement l'identité des serveurs (SEC-01, SEC-02).
3. **Banalisation des alertes et interruptions de service** : racine déployée par GPO, renouvellement automatique et supervision des expirations suppriment les fausses alertes (SEC-03). Une alerte restante redevient un vrai signal d'attaque.